

Aprendizaje de Máquinas

Control 3 Primavera 2024

Índice de Contenidos

Índice de Figuras

1.	Gráfico de los puntos	8
----	---------------------------------	---

Índice de Tablas

1.	Distancia euclidiana entre puntos, aproximada al primer decimal.	7
2.	Conjunto de datos para predecir si un estudiante aprueba o no en función de las horas de estudio y el color del cuaderno.	11

P1. (50 %) Preguntas Conceptuales

De las siguientes preguntas, debe elegir un máximo de **6 preguntas a contestar**, de donde debe escoger **dos preguntas de cada tópico (Clustering, Modelos basados en árboles y Redes Neuronales)**. Sea breve en la respuesta, se puede apoyar de dibujos. En caso de responder más de 2 preguntas por tópico, serán solo evaluadas las primeras 2 respuestas de cada tópico.

Clustering

1. ¿Qué rol juega la elección de la **métrica de similitud o distancia** en un algoritmo de clustering? Compare la **distancia Euclidiana** y otra métrica a su elección. Explique como afecta la **escala de los datos** en el rendimiento de los algoritmos.

Respuesta: La métrica determina qué tan “cercos” o “similares” se consideran dos puntos, por lo que *define la forma de los clusters*. Distintas métricas inducen distintas geometrías:

- **Distancia Euclidiana** (L_2): favorece clusters de forma aproximadamente esférica; es sensible a valores extremos.
- **Distancia Manhattan** (L_1): genera regiones con forma de rombo; es más robusta a outliers.

La **escala de las variables** afecta directamente estas distancias. Si una variable tiene valores numéricamente más grandes, dominará la métrica y “arrastrará” la asignación de los clusters. Por ello, algoritmos como k-means requieren **normalización o estandarización previa**.

$$d_{\text{euclid}}(x, y) = \sqrt{\sum_i (x_i - y_i)^2}, \quad d_{\text{man}}(x, y) = \sum_i |x_i - y_i|.$$

Un mal manejo de escalas puede deformar las distancias y degradar la calidad del clustering.

2. Describa el algoritmo **k-means** y de **DBSCAN**. ¿Cuáles son sus principales **limitaciones** y cómo se pueden abordar? ¿En qué casos conviene usar uno u el otro?

Respuesta:

k-means: algoritmo de partición que busca minimizar la suma de distancias cuadráticas dentro de cada cluster. Utiliza los *centroides* (promedios) como representaciones de cada grupo.

- a) Inicializa k centroides (aleatorios o mediante *k-means++*).
- b) Asigna cada punto a su centroide más cercano según distancia Euclidiana.
- c) Recalcula el centroide de cada cluster como el promedio de los puntos asignados.
- d) Itera asignación y actualización hasta que los centroides se estabilizan.

El método busca una solución que minimice:

$$\sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2.$$

Limitaciones de k-means y cómo abordarlas:

- **Clusters esféricos:** asume forma convexa; no captura estructuras complejas.
Solución: usar algoritmos alternativos como GMM, DBSCAN o k-medoids.
- **Sensibilidad a outliers:** los promedios se desplazan fuertemente.
Solución: limpiar datos, usar medoids o distancia Manhattan.
- **Fijar k a priori:** requiere decidir el número de clusters.
Solución: usar criterios como *elbow*, *silhouette* o información previa.
- **Dependencia de la inicialización:** puede converger a mínimos locales.
Solución: usar *k-means++* para inicializar.

DBSCAN: algoritmo basado en densidad que agrupa regiones con alta concentración de puntos y detecta ruido de forma natural.

- Cada punto puede ser **núcleo** (densidad alta), **frontera** o **ruido**.
- A partir de puntos núcleo, expande el cluster a todos los puntos alcanzables por densidad usando dos parámetros:
 ε (radio), **minPts** (mínimo de vecinos).
- Los puntos que no pertenecen a ninguna región suficientemente densa se etiquetan como *outliers*.

Limitaciones de DBSCAN y cómo abordarlas:

- **Elección de ε :** si el dataset tiene densidades muy distintas, un solo ε no funciona bien.
Solución: usar *OPTICS*.
- **Curse of dimensionality:** en alta dimensión todas las distancias se vuelven similares, dificultando medir densidades.
Solución: reducción de dimensionalidad previa (PCA, t-SNE, UMAP).
- **Sensibilidad a la métrica:** distintas métricas alteran la noción de vecindario.
Solución: elegir métrica adecuada según el tipo de variables.

¿Cuándo usar cada uno?

- **k-means:** datos numéricos, sin mucho ruido, con clusters aproximadamente esféricos y gran escala de muestras. Muy rápido y eficiente.
- **DBSCAN:** cuando se espera ruido, outliers o clusters de formas arbitrarias; cuando no se quiere fijar k ; cuando la estructura depende de la densidad.

3. Defina el concepto de **medoide** y compárelo con el de **centroide** en el contexto de clustering. ¿Utilizan los algoritmos de **clustering jerárquico** estas nociones?

Respuesta:

El **centroide** es el punto promedio de un cluster:

$$c = \frac{1}{n} \sum_{i=1}^n x_i,$$

por lo que *no necesariamente corresponde a un punto real del dataset* y es sensible a outliers.

El **medoide** es el punto del cluster que *minimiza la distancia total* a los demás puntos:

$$m = \arg \min_{x_j \in C} \sum_{x_i \in C} d(x_i, x_j).$$

Es robusto a ruido y siempre pertenece al conjunto de datos. Se utiliza en algoritmos como **k-medoids**.

Los algoritmos de **clustering jerárquico** normalmente no usan ni centroides ni medoides, sino **métricas de enlace** (*single, complete, average*) para medir similitud entre grupos. Solo variantes específicas como *centroid linkage* incorporan centroides.

4. Describa las diferencias entre un modelo **basado en densidad** como **DBSCAN** y un modelo basado en **distribuciones probabilísticas** como **GMM**.

Respuesta:

DBSCAN:

- Define clusters como regiones de *alta densidad*.
- No asume forma paramétrica de los clusters.
- Identifica ruido explícitamente.
- Produce clusters de forma arbitraria.

GMM (Gaussian Mixture Models):

- Asume que los datos provienen de una mezcla de distribuciones Gaussianas.
- Cada cluster tiene parámetros (μ_k, Σ_k) .
- Proporciona *probabilidades de pertenencia* (soft clustering).
- Requiere fijar el número de componentes.

Diferencias clave:

- a) DBSCAN no requiere suponer una distribución; GMM sí.
- b) DBSCAN detecta ruido; GMM asigna todos los puntos a una componente.
- c) GMM modela clusters elípticos; DBSCAN capta formas arbitrarias.

Modelos Basados en Árboles

1. ¿Qué son las **medidas de impureza** en los árboles de decisión? Compare **entropía** e **impureza de Gini**.

Respuesta:

Las **medidas de impureza** cuantifican el grado de mezcla de clases dentro de un nodo. Se utilizan para evaluar la calidad de una división: una buena división genera nodos “puros” (predominancia clara de una clase). Las dos medidas más comunes son:

- **Entropía** (utilizada en ID3, C4.5):

$$H(p_1, \dots, p_K) = - \sum_{k=1}^K p_k \log_2(p_k),$$

donde p_k es la proporción de la clase k en el nodo. Mide la incertidumbre general del nodo y penaliza fuertemente distribuciones uniformes.

- **Impureza de Gini** (utilizada en CART):

$$G(p_1, \dots, p_K) = 1 - \sum_{k=1}^K p_k^2.$$

Mide la probabilidad de clasificar incorrectamente un punto si se etiqueta al azar según la distribución del nodo.

Comparación:

- Ambas se minimizan cuando el nodo es puro.
 - La entropía penaliza ligeramente más las mezclas balanceadas; Gini es más rápida de calcular.
 - En la práctica suelen producir divisiones muy similares.
2. Explique el proceso de construcción de un árbol de decisión con el algoritmo **CART** ó con **ID3**, incluyendo su **criterio de división**. ¿En qué casos conviene utilizar un algoritmo u otro?

Respuesta:

Un árbol se construye recursivamente dividiendo el espacio en regiones cada vez más puras. El proceso es:

- a) Seleccionar una variable candidata y un punto de corte.
- b) Evaluar la calidad de la división usando una **medida de impureza**.
- c) Escoger la división que maximiza la **ganancia de información** (ID3) o minimiza la **impureza ponderada** (CART).
- d) Repetir el proceso en cada nodo hijo hasta cumplir un criterio de parada (profundidad, número mínimo de muestras, etc.).

ID3: usa **entropía** y **ganancia de información**. Funciona bien en tareas puramente categóricas.

$$\text{InfoGain} = H(\text{padre}) - \sum_j \frac{n_j}{n} H(\text{hijo}_j)$$

CART: utiliza **impureza de Gini** y genera siempre divisiones *binarias*. También permite regresión usando MSE.

$$\text{GiniSplit} = \sum_j \frac{n_j}{n} G(\text{hijo}_j)$$

¿Cuándo usar cada uno?

- **ID3:** datos categóricos, cuando se prefiere interpretar particiones multivaluadas.
- **CART:** datos mixtos, divisiones binarias más controlables, problemas de regresión.

3. Explique el concepto de **boosting** en el contexto de árboles de decisión y cómo **AdaBoost** ajusta los **pesos de las observaciones**.

Respuesta:

El **boosting** combina muchos clasificadores débiles (por ejemplo, árboles pequeños tipo *stumps*) para producir un clasificador fuerte. Cada clasificador se entrena para corregir los errores del anterior, asignando mayor peso a las observaciones difíciles.

AdaBoost funciona así:

- a) Inicializa pesos uniformes $w_i = \frac{1}{n}$.
- b) Entrena un árbol débil usando estos pesos.
- c) Calcula el error ponderado:

$$\varepsilon_t = \sum_{i=1}^n w_i \cdot \mathbb{I}(h_t(x_i) \neq y_i).$$

- d) Asigna un peso al clasificador:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right).$$

- e) **Actualiza los pesos** aumentando los de las observaciones mal clasificadas:

$$w_i \leftarrow w_i \exp(\alpha_t \mathbb{I}(h_t(x_i) \neq y_i)).$$

- f) Normaliza los pesos y repite.

El modelo final es una combinación ponderada:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right).$$

- ¿Qué **ventajas** y **desventajas** tienen los modelos basados en árboles frente a otros enfoques de **aprendizaje supervisado**?

Respuesta:

Ventajas:

- Altamente interpretables; reglas explícitas.
- Capturan interacciones no lineales entre variables.
- Requieren poca preparación de datos (sin normalizar escalas).
- Funcionan bien con datos categóricos o mixtos.

Desventajas:

- Tienden a sobreajustar si no se podan.
- Un único árbol es inestable: pequeñas variaciones del dataset pueden cambiar mucho el árbol.
- Límites de decisión en forma de rectángulos: menos flexibles que métodos kernel o redes neuronales.
- Menor rendimiento predictivo comparado con modelos ensemble (Random Forest, Boosting).

Redes Neuronales

- Explique qué es una **red neuronal feed-forward** y profundice en el papel de las **funciones de activación**.
- ¿Qué establece el **Teorema de Aproximación Universal** sobre las redes neuronales? ¿Qué limitaciones prácticas tiene este teorema?
- ¿Equivale una **red neuronal de una capa** (sin capa oculta) y con **función de activación sigmoide** para clasificación a una **regresión logística**?
- ¿Es la **función de costos** siempre **diferenciable** respecto a los parámetros? ¿Es siempre **convexa** con respecto a los parámetros? ¿Qué influye en que las redes neuronales sean o no lo anterior?

P2. (15 %) Clustering

Considere los siguientes puntos en el plano:

$$(1, 2), (2, 2), (2, 3), (8, 7), (8, 8), (10, 3), (1, 3), (2, 4) \in \mathbb{R}^2.$$

Donde la **distancia euclidiana** entre los puntos (aproximada al primer decimal) se encuentra en la **Tabla 1** y un gráfico de los puntos en la **Figura 1**.

Tabla 1: Distancia euclidiana entre puntos, aproximada al primer decimal.

	(1, 2)	(2, 2)	(2, 3)	(8, 7)	(8, 8)	(10, 3)	(1, 3)	(2, 4)
(1, 2)	0.0	1.0	1.4	8.6	9.2	9.1	1.0	2.2
(2, 2)	1.0	0.0	1.0	7.8	8.5	8.1	1.4	2.0
(2, 3)	1.4	1.0	0.0	7.2	7.8	8.0	1.0	1.0
(8, 7)	8.6	7.8	7.2	0.0	1.0	4.5	8.1	6.7
(8, 8)	9.2	8.5	7.8	1.0	0.0	5.4	8.6	7.2
(10, 3)	9.1	8.1	8.0	4.5	5.4	0.0	9.0	8.1
(1, 3)	1.0	1.4	1.0	8.1	8.6	9.0	0.0	1.4
(2, 4)	2.2	2.0	1.0	6.7	7.2	8.1	1.4	0.0

1. Aplique el algoritmo de **DBSCAN** sobre este conjunto de datos utilizando $\varepsilon = 2$ como radio máximo para una vecindad y **MinPts** = 3 como el número mínimo de vecinos para considerar una zona como densa. Explícite qué puntos quedan como **puntos core** y cuáles como **ruido**.

Respuesta:

Vecindades (radio $\varepsilon = 2$, contando el punto mismo).

Usando la tabla de distancias (aproximada a 0.1) la vecindad $\mathcal{N}_\varepsilon(P_i) = \{P_j : d(P_i, P_j) \leq 2\}$ es:

$$\begin{aligned}
 \mathcal{N}_\varepsilon(P_1) &= \{P_1, P_2, P_3, P_7\} \quad (\text{card} = 4) \\
 \mathcal{N}_\varepsilon(P_2) &= \{P_1, P_2, P_3, P_7, P_8\} \quad (\text{card} = 5) \\
 \mathcal{N}_\varepsilon(P_3) &= \{P_1, P_2, P_3, P_7, P_8\} \quad (\text{card} = 5) \\
 \mathcal{N}_\varepsilon(P_4) &= \{P_4, P_5\} \quad (\text{card} = 2) \\
 \mathcal{N}_\varepsilon(P_5) &= \{P_4, P_5\} \quad (\text{card} = 2) \\
 \mathcal{N}_\varepsilon(P_6) &= \{P_6\} \quad (\text{card} = 1) \\
 \mathcal{N}_\varepsilon(P_7) &= \{P_1, P_2, P_3, P_7, P_8\} \quad (\text{card} = 5) \\
 \mathcal{N}_\varepsilon(P_8) &= \{P_2, P_3, P_7, P_8\} \quad (\text{card} = 4)
 \end{aligned}$$

Clasificación core / border / ruido.

Recordemos: un punto es *core* si $|\mathcal{N}_\varepsilon(P)| \geq \text{MinPts} = 3$. Un punto no-core que pertenece a la vecindad de algún core es *border*; si no pertenece a la vecindad de ningún core se clasifica como *ruido*.

De las cardinalidades arriba:

- **Core points** ($|\mathcal{N}| \geq 3$):

$$P_1, P_2, P_3, P_7, P_8.$$

(es decir, los puntos con coordenadas alrededor de $x \in \{1, 2\}, y \in \{2, 3, 4\}$).

- **No-core:** P_4, P_5, P_6 .

- ¿Son border o ruido? comprobamos si alguno de P_4, P_5, P_6 está en la vecindad de algún core:

- P_4, P_5 sólo se tienen entre sí en su vecindad ($\mathcal{N}_\varepsilon(P_4) = \{P_4, P_5\}$); no pertenecen a la vecindad de ningún core.
- P_6 está aislado ($\mathcal{N}_\varepsilon(P_6) = \{P_6\}$).

Por tanto P_4, P_5, P_6 **no** son border: son **ruido** bajo estos parámetros.

Resultado DBSCAN (agrupamiento):

DBSCAN expande clusters a partir de puntos core y todos los puntos *density-reachable* desde ellos. Aquí los cores $\{P_1, P_2, P_3, P_7, P_8\}$ forman un único cluster (todos son mutua o transitivamente alcanzables por vecindades), y no hay bordes adicionales. Por lo tanto:

Cluster 1 (core cluster) = $\{P_1, P_2, P_3, P_7, P_8\}$.

Y

Ruido = $\{P_4, P_5, P_6\}$.

2. Aplique un algoritmo de **clustering jerárquico** con **Enlace Simple** y otro con **Enlace Completo**.

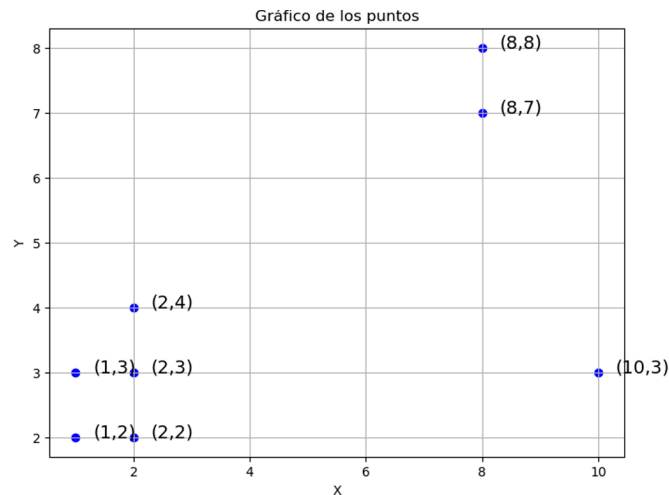


Figura 1: Gráfico de los puntos

Respuesta:

Usamos la matriz de distancias euclidianas (la proporcionada en el enunciado). A continuación se da la **secuencia de fusiones** (pares de clusters que se unen y la distancia en la que se produce la fusión) para cada criterio de enlace.

Notación:

escribimos la fusión como $\text{ClusterA} \mid \text{ClusterB} \quad @ d$ indicando que los dos clusters se unen a distancia d .

Enlace *Single* (mínima distancia entre clusters). Fusiones en orden creciente de distancia:

1. $P_1 \mid P_2 \quad @ d = 1.0$.
2. $P_3 \mid P_7 \quad @ d = 1.0$.
3. $P_4 \mid P_5 \quad @ d = 1.0$.
4. $P_8 \mid (P_3 + P_7) \quad @ d = 1.0$.
(porque $d(P_8, P_3) = 1.0$ es la mínima distancia entre P_8 y el par $\{P_3, P_7\}$).
5. $(P_1 + P_2) \mid (P_8 + P_3 + P_7) \quad @ d = 1.0$.
(con esto se forma el gran subcluster de los cinco puntos $\{P_1, P_2, P_3, P_7, P_8\}$ a distancia 1.0).
6. $P_6 \mid (P_4 + P_5) \quad @ d = 4.5$.
(la mínima distancia entre $P_6 = (10, 3)$ y el par $\{P_4, P_5\}$ es $d(P_6, P_4) = 4.5$).
7. $(P_1 + P_2 + P_3 + P_7 + P_8) \mid (P_6 + P_4 + P_5) \quad @ d = 6.7$.

Interpretación / particiones típicas (single linkage):

- Si cortamos el dendrograma por debajo de $d = 4.5$ (por ejemplo a $d = 2$) obtenemos tres clusters:

$$C_1 = \{P_1, P_2, P_3, P_7, P_8\}, \quad C_2 = \{P_4, P_5\}, \quad C_3 = \{P_6\}.$$

- Si pedimos $k = 2$ clusters (corte entre $d = 4.5$ y $d = 6.7$), obtenemos la partición:

$$\{P_1, P_2, P_3, P_7, P_8\} \quad \text{y} \quad \{P_4, P_5, P_6\}.$$

Enlace *Complete* (máxima distancia entre clusters). Fusiones en orden creciente de la distancia de enlace completo:

1. $P_1 \mid P_2 \quad @ d = 1.0$.
2. $P_3 \mid P_7 \quad @ d = 1.0$.
3. $P_4 \mid P_5 \quad @ d = 1.0$.
4. $P_8 \mid (P_3 + P_7) \quad @ d = 1.4$.
(en enlace completo la distancia entre P_8 y el cluster $\{P_3, P_7\}$ es $\max\{d(P_8, P_3), d(P_8, P_7)\} = \max\{1.0, 1.4\} = 1.4$.)
5. $(P_1 + P_2) \mid (P_8 + P_3 + P_7) \quad @ d = 2.2$.
(en enlace completo la distancia entre estos dos subclusters es la mayor distancia entre sus puntos, que resulta ser 2.2).
6. $P_6 \mid (P_4 + P_5) \quad @ d = 5.4$.

$$7. (P_1 + P_2 + P_3 + P_7 + P_8) \mid (P_6 + P_4 + P_5) \quad @ d = 9.2.$$

Interpretación / particiones típicas (complete linkage):

- Para un corte por debajo de $d = 2.2$ (por ejemplo $d = 1.5$) obtenemos los mismos clusters finos iniciales: $\{P_1, P_2\}, \{P_3, P_7, P_8\}, \{P_4, P_5\}, \{P_6\}$.
- Para $k = 3$ es razonable la partición (corte entre $d = 2.2$ y $d = 5.4$):

$$C_1 = \{P_1, P_2, P_3, P_7, P_8\}, \quad C_2 = \{P_4, P_5\}, \quad C_3 = \{P_6\}.$$

- Para $k = 2$ (corte entre $d = 5.4$ y $d = 9.2$) se obtiene la misma partición en dos grupos que en enlace simple:

$$\{P_1, P_2, P_3, P_7, P_8\} \quad \text{y} \quad \{P_4, P_5, P_6\}.$$

Comentarios comparativos entre enlaces (intuición):

- **Single linkage** tiende a unir mediante la mínima distancia (produce cadenas y es sensible a puntos puente). En nuestro conjunto esto hace que los cinco puntos cercanos (los de la esquina inferior izquierda del plano) se agrupen muy pronto (a distancia 1.0).
- **Complete linkage** exige que todos los pares entre clusters sean cercanos (usa la máxima distancia). Esto genera clusters más compactos; por eso algunas fusiones que en single ocurren a $d = 1.0$ en complete suceden a distancias mayores (1.4, 2.2, ...).
- Sin embargo, para particiones gruesas (por ejemplo $k = 2$) ambos métodos sugieren la misma separación natural: el grupo de cinco puntos cercanos $\{P_1, P_2, P_3, P_7, P_8\}$ frente al subgrupo $\{P_4, P_5, P_6\}$ (los puntos aislados en la esquina superior derecha y el punto P_6).

P3. (25 %) Árboles

En un determinado curso escolar, se mide si un estudiante aprueba o no un curso en base a sus **horas de estudio** y el **color de su cuaderno**, esto se ve reflejado en la **Tabla 2**.

Tabla 2: Conjunto de datos para predecir si un estudiante aprueba o no en función de las horas de estudio y el color del cuaderno.

Horas de Estudio	Color del Cuaderno	Aprueba
> 5	Azul	Sí
<= 5	Azul	No
> 5	Rojo	Sí
<= 5	Rojo	No
> 5	Verde	Sí
<= 5	Verde	Sí

1. Cree un **árbol de decisión binario** el cual la primera división la haga por las **horas de estudio**. Aplique su propio criterio para realizar las divisiones.

Respuesta:

Elegimos la partición natural $\text{Horas} > 5$ frente a $\text{Horas} \leq 5$. Observaciones:

- $\text{Horas} > 5$: tres ejemplos (Azul, Rojo, Verde) — $\{\text{Sí}, \text{Sí}, \text{Sí}\} \Rightarrow$ **puro (Sí)**.
- $\text{Horas} \leq 5$: tres ejemplos (Azul, Rojo, Verde) — $\{\text{No}, \text{No}, \text{Sí}\} \Rightarrow$ mezcla.

Un árbol binario sencillo que respeta la consigna (primera división por Horas) es:

Nodo raíz: Horas > 5?

$$\left\{ \begin{array}{l} \text{Sí} \Rightarrow \text{Clase} = \text{Sí} \\ \text{No} \Rightarrow (\text{dividir por Color}) \\ \left\{ \begin{array}{l} \text{Color} = \text{Verde} \Rightarrow \text{Clase} = \text{Sí} \\ \text{Color} = \text{Azul o Rojo} \Rightarrow \text{Clase} = \text{No} \end{array} \right. \end{array} \right. \quad \begin{array}{l} (3/3) \\ \\ (1/3) \\ (2/3) \end{array}$$

(Explicación: en la rama $\text{Horas} > 5$ todos aprueban, por eso es hoja pura. En la rama ≤ 5 sólo el caso “Verde” aprueba, por tanto un test binario por color (Verde | no Verde) separa correctamente.)

2. Cree un **árbol de decisión binario** el cual la primera división la haga por el **color del cuaderno**. Aplique su propio criterio para realizar las divisiones.

Respuesta:

Elegimos una partición binaria sobre el atributo categórico de tres valores; una elección natural es separar Verde frente a $\{\text{Azul}, \text{Rojo}\}$. Observaciones:

- $\text{Color} = \text{Verde}$: dos ejemplos (uno con $\text{Horas} > 5$, otro con $\text{Horas} \leq 5$) — $\{\text{Sí}, \text{Sí}\} \Rightarrow$ **puro (Sí)**.
- $\text{Color} \in \{\text{Azul}, \text{Rojo}\}$: cuatro ejemplos — $\{\text{Sí}, \text{No}, \text{Sí}, \text{No}\}$ i.e. 2 Sí y 2 No (mezcla).

Un árbol binario con primera división por color:

$$\begin{array}{l} \text{Nodo raíz: Color = Verde?} \\ \left\{ \begin{array}{l} \text{Sí} \Rightarrow \text{Clase} = \text{Sí} \\ \text{No} \Rightarrow (\text{dividir por Horas}) \end{array} \right. \quad (2/2) \\ \left\{ \begin{array}{l} \text{Horas} > 5 \Rightarrow \text{Clase} = \text{Sí} \\ \text{Horas} \leq 5 \Rightarrow \text{Clase} = \text{No} \end{array} \right. \quad (2/2) \end{array}$$

(Explicación: para los no-Verde, la separación por Horas distingue perfectamente Sí vs No.)

3. Sin realizar ningún cálculo, **justifique** por qué alguno de los dos árboles de decisión tiene que tener **mayor ganancia de información** que el otro.

Respuesta:

La ganancia de información mide la reducción de incertidumbre (entropía) al aplicar una partición. En ambos árboles la primera división produce una **hoja pura** y una rama mixta, pero el **tamaño** de la hoja pura es distinto:

- División por **Horas**: la hoja pura ($\text{Horas} > 5$) contiene 3 observaciones todas Sí.
- División por **Color** (Verde vs otros): la hoja pura (Verde) contiene sólo 2 observaciones, ambas Sí.

Una hoja pura de mayor tamaño elimina más incertidumbre (más ejemplos quedan explicados de forma determinista). Por tanto la partición que produce la **hoja pura más grande** (en este caso $\text{Horas} > 5$ con 3 ejemplos) reduce la entropía más y, sin hacer cálculos, debe proporcionar **mayor ganancia de información** que la partición por color (que produce hoja pura de tamaño 2).

4. Calcule la **ganancia de información** solo de la primera división del árbol de decisión por las **horas de estudio** según el **Criterio de Entropía**.

Respuesta:

Entropía del nodo raíz (antes de dividir).

En el nodo raíz hay 6 ejemplos: 4 Sí y 2 No. Las probabilidades son

$$p_{\text{Sí}} = \frac{4}{6}, \quad p_{\text{No}} = \frac{2}{6}.$$

La entropía (base 2) del nodo raíz es

$$H(\text{root}) = -\frac{4}{6} \log_2 \left(\frac{4}{6} \right) - \frac{2}{6} \log_2 \left(\frac{2}{6} \right).$$

Calculamos numéricamente (mostrando pasos):

$$\frac{4}{6} \approx 0.6667, \quad \frac{2}{6} \approx 0.3333.$$

$$H(\text{root}) = -0.6667 \log_2(0.6667) - 0.3333 \log_2(0.3333) \approx 0.9183 \text{ bits.}$$

Entropías de las ramas tras dividir por Horas:

- Rama Horas > 5 : 3 ejemplos, todos Sí. Entonces

$$H(\text{Horas} > 5) = 0 \quad (\text{nodo puro}).$$

- Rama Horas ≤ 5 : 3 ejemplos, 1 Sí y 2 No. Probabilidades: $p_{\text{Sí}} = \frac{1}{3} \approx 0.3333$, $p_{\text{No}} = \frac{2}{3} \approx 0.6667$.

$$H(\text{Horas} \leq 5) = -\frac{1}{3} \log_2\left(\frac{1}{3}\right) - \frac{2}{3} \log_2\left(\frac{2}{3}\right) \approx 0.9183 \text{ bits.}$$

(Obs.: numéricamente resulta igual a la entropía del nodo raíz en este caso parcial.)

Entropía promedio posterior a la división (weighted entropy).

Cada rama tiene 3 ejemplos sobre 6, por lo que los pesos son $3/6 = 0.5$ para cada rama:

$$H_{\text{after}} = \frac{3}{6} \cdot H(\text{Horas} > 5) + \frac{3}{6} \cdot H(\text{Horas} \leq 5) = 0.5 \cdot 0 + 0.5 \cdot 0.9183 = 0.45915 \text{ bits.}$$

Ganancia de información (Information Gain).

$$\text{Gain} = H(\text{root}) - H_{\text{after}} \approx 0.91830 - 0.45915 = 0.45915 \text{ bits.}$$

$\text{Gain}(\text{Horas} > 5) \approx 0.4591 \text{ bits.}$

Conclusión breve: El árbol cuya primera división es por Horas crea una hoja pura más grande (3 ejemplos) y, por lo tanto, proporciona mayor reducción de entropía que la división por Color (que crea una hoja pura de tamaño 2). Numéricamente la ganancia de información de la primera división por Horas es ≈ 0.4591 bits.

P4. (10 %) Redes neuronales

1. Considere una **red neuronal feed-forward** $\mathcal{N}_2 : \mathbb{R}^2 \rightarrow \mathbb{R}$ con una sola capa oculta con **dos neuronas** con una **función de activación ReLU** y **capa de salida lineal**. Pruebe que existen pesos tales que $\mathcal{N}_2(x_1, x_2) = -x_2$, para todo $(x_1, x_2) \in \mathbb{R}^2$.